

Dynamic semantics

Overview

- ▶ defines the **behavior** of a program at **runtime** (= when it is executed)
- ▶ implemented by an **interpreter** or a **compiler**

Reminder

- ▶ an **interpreter** directly executes the program; in other words, the program is executed on a **virtual machine**
- ▶ a **compiler** “translates” the program into “executable” lower-level code

Dynamic semantics

Typical actions performed during a program execution

- ▶ the values of expressions are computed
- ▶ I/O is performed (e.g., reading/writing files, network interaction)
- ▶ the memory is modified
 - ▶ new variables are declared in the current level
 - ▶ a block of statements is entered and its nested level of declarations added
 - ▶ a block of statements is exited and its level of declarations removed
 - ▶ the content of variables is modified
 - ▶ ...

Remark: errors can occur at runtime; they are called **dynamic errors**

Definition of the dynamic semantics

both with semantic rules and F# definition, as for the static semantics

Variable declarations in the dynamic semantics

Dynamic environments

- ▶ variable declarations are managed with **dynamic environments**
- ▶ dynamic environments are similar to static environments, except for two differences

Differences between dynamic and static environments

- ▶ variables are associated with **values** (not with types)
- ▶ the values of variables can be **updated** with the assignment statement

Remark: in static environments the type of a variable **cannot** be updated

once a variable has been declared, its type is **not modifiable**

Semantic rules: formalization

The dynamic environment ρ is a sequence of maps associating variables with values v

Values:

$$v ::= i \mid b \mid (v, v)$$

Remark: values include integers i , booleans b , and pairs of values (v_1, v_2)

The evaluation relation for expressions is denoted by:

$$\rho \vdash e \Rightarrow v$$

which means that expression e evaluates to value v in the environment ρ

Semantic rules: expressions

E-INT

$$\frac{}{\rho \vdash i \Rightarrow i}$$

E-BOOL

$$\frac{}{\rho \vdash b \Rightarrow b}$$

E-VAR

$$\frac{\text{lookup}(x, \rho) = v}{\rho \vdash x \Rightarrow v}$$

E-ADD

$$\frac{\rho \vdash e_1 \Rightarrow i_1 \quad \rho \vdash e_2 \Rightarrow i_2 \quad i = i_1 + i_2}{\rho \vdash e_1 + e_2 \Rightarrow i}$$

E-MUL

$$\frac{\rho \vdash e_1 \Rightarrow i_1 \quad \rho \vdash e_2 \Rightarrow i_2 \quad i = i_1 * i_2}{\rho \vdash e_1 * e_2 \Rightarrow i}$$

E-MINUS

$$\frac{\rho \vdash e \Rightarrow i \quad i' = -i}{\rho \vdash -e \Rightarrow i'}$$

Semantic rules: expressions (cont.)

E-EQ

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow v_2 \quad b = (v_1 == v_2)}{\rho \vdash e_1 == e_2 \Rightarrow b}$$

E-AND

$$\frac{\rho \vdash e_1 \Rightarrow b_1 \quad \rho \vdash e_2 \Rightarrow b_2 \quad b = b_1 \&\& b_2}{\rho \vdash e_1 \&\& e_2 \Rightarrow b}$$

E-NOT

$$\frac{\rho \vdash e \Rightarrow b \quad b' = !b}{\rho \vdash !e \Rightarrow b'}$$

E-PAIR

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow v_2}{\rho \vdash (e_1, e_2) \Rightarrow (v_1, v_2)}$$

E-FST

$$\frac{\rho \vdash e \Rightarrow (v_1, v_2)}{\rho \vdash \text{fst } e \Rightarrow v_1}$$

E-SND

$$\frac{\rho \vdash e \Rightarrow (v_1, v_2)}{\rho \vdash \text{snd } e \Rightarrow v_2}$$

Semantic rules: statements and output

The evaluation relation for statements is denoted by:

$$\langle \rho, o \rangle \vdash s \Rightarrow \langle \rho', o' \rangle$$

which means that the execution of statement s with respect to the environment ρ and output o produces a new environment ρ' and a new output o'

Remark: o is modeled as a sequence of characters (a string); in practice, it corresponds to the standard output stream

Semantics rules: statements

S-ASSIGN

$$\frac{\rho \vdash e \Rightarrow v \quad \rho' = \text{update}(x, v, \rho)}{\langle \rho, o \rangle \vdash x = e \Rightarrow \langle \rho', o \rangle}$$

S-VAR

$$\frac{\rho \vdash e \Rightarrow v \quad \rho' = \text{dec}(x, v, \rho)}{\langle \rho, o \rangle \vdash \text{var } x = e \Rightarrow \langle \rho', o \rangle}$$

S-PRINT

$$\frac{\rho \vdash e \Rightarrow v \quad o' = o + \text{toString}(v) + "\backslash n"}{\langle \rho, o \rangle \vdash \text{print } e \Rightarrow \langle \rho, o' \rangle}$$

S-ASSERT

$$\frac{\rho \vdash e \Rightarrow \text{true}}{\langle \rho, o \rangle \vdash \text{assert } e \Rightarrow \langle \rho, o \rangle}$$

Semantic rules: conditional statement

S-IF-THEN-ELSE-TRUE

$$\frac{\rho \vdash e \Rightarrow \text{true} \quad \langle \rho, o \rangle \vdash \text{block}_{\text{then}} \Rightarrow \langle \rho', o' \rangle}{\langle \rho, o \rangle \vdash \text{if } (e) \text{ block}_{\text{then}} \text{ else } \text{block}_{\text{else}} \Rightarrow \langle \rho', o' \rangle}$$

S-IF-THEN-ELSE-FALSE

$$\frac{\rho \vdash e \Rightarrow \text{false} \quad \langle \rho, o \rangle \vdash \text{block}_{\text{else}} \Rightarrow \langle \rho', o' \rangle}{\langle \rho, o \rangle \vdash \text{if } (e) \text{ block}_{\text{then}} \text{ else } \text{block}_{\text{else}} \Rightarrow \langle \rho', o' \rangle}$$

Semantic rules: sequences and blocks

SSEQ-EMPTY

$$\frac{}{\langle \rho, o \rangle \vdash \epsilon \Rightarrow \langle \rho, o \rangle}$$

SSEQ-NONEMPTY

$$\frac{\langle \rho, o \rangle \vdash s \Rightarrow \langle \rho', o' \rangle \quad \langle \rho', o' \rangle \vdash \bar{s} \Rightarrow \langle \rho'', o'' \rangle}{\langle \rho, o \rangle \vdash s; \bar{s} \Rightarrow \langle \rho'', o'' \rangle}$$

B-BLOCK

$$\frac{\langle \{ \} :: \rho, o \rangle \vdash \bar{s} \Rightarrow \langle m :: \rho', o' \rangle}{\langle \rho, o \rangle \vdash \{ \bar{s} \} \Rightarrow \langle \rho', o' \rangle}$$

Semantic rules: programs

Program execution starts from an environment with an empty global scope and an empty output string.

$$\frac{\text{P-PROG} \quad \langle \{ \} :: [], " " \rangle \vdash \bar{s} \Rightarrow \langle \rho, o \rangle}{\vdash \text{program } \bar{s} \Rightarrow o}$$

- ▶ the initial environment is a list containing an empty map: $\{ \} :: []$
- ▶ the final result of program execution is the output string o
- ▶ any dynamic errors (such as failed `assert` statements or undeclared variables) interrupt the execution: no derivation tree can be built

Dynamic semantics: auxiliary functions

► Variable Lookup:

$\text{lookup} : \text{Variable} \rightarrow \text{DynamicEnv} \rightarrow \text{Value}$
retrieves the value of a variable from the environment

► Variable Declaration:

$\text{dec} : \text{Variable} \rightarrow \text{Value} \rightarrow \text{DynamicEnv} \rightarrow \text{DynamicEnv}$
introduces a new variable in the current scope level

Remark: the definitions of `lookup` and `dec` are **analogous** to those used in the static semantics, but they operate on values and dynamic environments instead of types and static environments

Dynamic semantics: environment update

Unlike the static semantics, the dynamic semantics includes an `update` function to modify variables that are **already declared**

`update` : `Variable` × `Value` × `DynamicEnv` → `DynamicEnv`

$$\text{update}(x, v, m :: \rho) = \begin{cases} (m[x \mapsto v]) :: \rho & \text{if } x \in \text{dom}(m) \\ m :: \text{update}(x, v, \rho) & \text{if } x \notin \text{dom}(m) \end{cases}$$

`update`(`x`, `v`, `[]`) is undefined

$m[x \mapsto v]$ is the map that is identical to m except for the variable x , which is now associated with the value v

Remark: if x is not found in any level of the environment, the function is undefined (this corresponds to the `UndeclaredVariable` error)

Dynamic environments in F#

Technical details

- ▶ definition of value

```
type value =  
    | IntValue of int | BoolValue of bool | PairValue of value * value
```

- ▶ definition of dynamic environment and output

```
type variable = Name of string  
type scopeLevel = Map<variable, value>  
type dynamicEnv = scopeLevel list  
type output = string
```

- ▶ some basic environments

- ▶ a level with no declarations

```
let emptyLevel: scopeLevel = Map.empty
```

- ▶ the initial environment before the program will be executed

```
let initialEnv: dynamicEnv = [ emptyLevel ]
```

Dynamic environments in F#

Example with two nested scope levels

a list of two single level declarations

```
env = [map [(Name "x", IntValue 1); (Name "y", BoolValue false)]; map [(Name "x", BoolValue true); (Name "z", IntValue 2)]]
```

declarations at level 1: x contains 1 and y contains false

declarations at level 0 (=top-level): x contains true and z contains 2

Dynamic environments and variable update in F#

- ▶ (already seen) variable assignment changes the dynamic environment
- ▶ (new) it can change the value of a variable declared at any level

Dynamic environments and variable update in F#

Example

```
var x=1;  
if (!(x==0)) {var y=2;x=x+y};  
print x // prints 3
```

the environment **just before** the execution of `x=x+y` is

```
[map [(Name "y", IntValue 2)]; map [(Name "x", IntValue 1)]]
```

the environment **just after** the execution of `x=x+y` is

```
[map [(Name "y", IntValue 2)]; map [(Name "x", IntValue 3)]]
```

the environment **just before** the execution of `print x` is

```
[map [(Name "x", IntValue 3)]]
```

Dynamic environments in F#

Functions for handling the dynamic environment

Similar functions used for the static semantics

```
enterLevel: dynamicEnv -> dynamicEnv  
exitLevel: dynamicEnv -> dynamicEnv  
lookup: variable -> dynamicEnv -> value  
dec: variable -> value -> dynamicEnv -> dynamicEnv
```

Dynamic environments: function update

for the dynamic semantics an additional function for updating variables is needed

Definition

`update: variable -> value -> dynamicEnv -> dynamicEnv`

- ▶ function `update` searches in the environment the definition of a variable
- ▶ the search starts from the most nested level and continues in the outer levels until the variable is found or no levels are left
- ▶ if the variable is found, then its associated value is updated with the new value
- ▶ otherwise an exception is raised

Definition of the dynamic semantics in F#

Functions which define the dynamic semantics

Remark: all functions may raise a **dynamic error**

```
executeProg : prog -> output
```

executes a program starting from `initialEnv` (see page 14) and returns the corresponding output

```
executeStmtSeq:
```

```
dynamicEnv * output -> stmtSeq -> dynamicEnv * output
```

executes a statement sequence with respect to an environment and an output and returns an environment and an output

```
executeBlock:
```

```
dynamicEnv * output -> block -> dynamicEnv * output
```

executes a block of statements with respect to an environment and output and returns an environment and output

Definition of the dynamic semantics in F#

Functions which define the dynamic semantics

```
executeStmt :  
    dynamicEnv * output -> stmt -> dynamicEnv * output
```

executes a statement with respect to an environment and a output
and returns an environment and an output

```
evalExp : dynamicEnv -> exp -> value
```

computes and returns the value of an expression in a environment

Definition of the dynamic semantics is F#

Remark

note the difference between

```
executeStmtSeq: ... -> dynamicEnv * output  
executeBlock: ... -> dynamicEnv * output
```

and

```
typecheckStmtSeq: ... -> unit  
typecheckBlock: ... -> unit
```

except for the output, which is only needed for the dynamic semantics

- ▶ `executeStmtSeq` and `executeBlock` return an environment because execution in nested levels can change outer levels
- ▶ `typecheckStmtSeq` and `typecheckBlock` do not return an environment because typechecking in nested levels cannot change outer levels

Definition of the dynamic semantics

The dynamic semantics is specified by semantic rules

Rules defined on the AST

- ▶ semantic rules defined for each type of node of the AST
- ▶ execution of a program = a visit of its AST

Implementation of a selection of the semantic rules

Variable

expression Variable var



if lookup var env = val

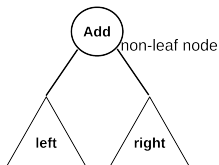
then Variable var has value val in the environment env

else it is undefined in env and a dynamic error is raised

Implementation of a selection of the semantic rules

Addition

expression `Add(left, right)`



if `left` and `right` have values `val1` and `val2` in `env`
and `val1+val2=val`

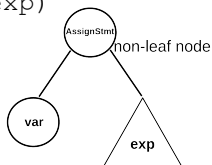
then `Add(left, right)` has value `val` in `env`

else it is **undefined** in `env` and a dynamic error is raised

Implementation of a selection of the semantic rules

Assignement

statement `AssignStmt (var, exp)`



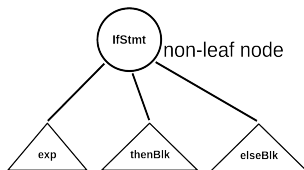
if `exp` has value `val` in `env`,
and update `var val env = env2`

then `AssignStmt (var, exp)` produces the environment `env2`

else it is **undefined** in `env` and a dynamic error is raised

Implementation of a selection of the semantic rules

if-then-else statement

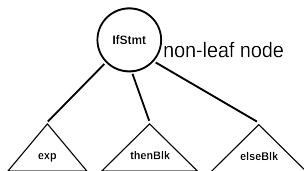


if `exp` has value **true** in `env`, and the execution of `thenBlk` in `env` and `out` produces `env2` and `out2`

then `IfStmt(exp, thenBlk, elseBlk)` produces the environment `env2` and the output `out2`

Implementation of a selection of the semantic rules

if-then-else statement (cont.)



else if `exp` has value **false** in `env`, and the execution of `elseBlk` in `env` and `out` produces `env3` and `out3`

then `IfStmt (exp, thenBlk, elseBlk)` produces the environment `env3` and the output `out3`

else `IfStmt (exp, thenBlk, elseBlk)` is undefined and a dynamic error is raised